

Chapitre 7: Prédiction, taux d'erreur, sélection de modèles et régularisation

1 Taux d'erreur

Pour faciliter la compréhension des notions présentées, on limite la matière pour le moment à la régression linéaire. Les généralisations à la régression logistique seront abordées plus tard.

Considérons le jeu de données `Auto` de la library `ISLR` du logiciel R. Ce jeu de données contient 392 observations. La variable réponse est `mpg`, la distance moyenne parcourue (en miles) par la voiture avec un gallon de carburant. On s'intéresse à une seule variable explicative, soit la puissance de la voiture horsepower.

On veut choisir un parmi les modèles suivants:

$$\text{Modèle 1: } Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$

$$\text{Modèle 2: } Y_i = \beta_0 + \beta_1 X_i + \beta_2 \log(X_i) + \epsilon_i$$

$$\text{Modèle 3: } Y_i = \beta_0 + \beta_1 X_i + \beta_2 X_i^2 + \epsilon_i$$

$$\text{Modèle 4: } Y_i = \beta_0 + \beta_1 X_i + \beta_2 X_i^2 + \beta_3 X_i^3 + \epsilon_i$$

Pour tous ces modèles, l'indice i varie de 1 à $n = 392$ et on suppose que $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$.

Pour chaque modèle, on estime les coefficients β_j par la méthode des moindres carrés et on calcule les valeurs prédites $\hat{Y}_i, i = 1, \dots, n$. Par exemple, pour le modèle 1, on a $\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1 X_i$. Par la suite, on calcule la somme des carrés des résidus

$$SSE = \sum_{i=1}^n (Y_i - \hat{Y}_i)^2.$$

Pour choisir entre les 4 modèles listés plus haut, on peut calculer des critères comme l'*AIC* ou le *BIC* ou effectuer des tests d'hypothèses pour les modèles emboîtés. Toutes ces procédures reviennent à comparer des *SSEs* des différents modèles. Ces méthodes cherchent donc à identifier le modèle avec le meilleur ajustement aux données.

Supposer maintenant qu'on a une nouvelle valeur de la variable explicative X_0 et on veut sélectionner le modèle qui prédit la variable réponse Y_0 avec la meilleure précision. Autrement dit, on veut identifier le modèle qui minimise $E[(Y_0 - \hat{Y}_0)^2]$.

Les deux critères (adéquation et précision de la prédiction) sont différents et peuvent même être contradictoires. Si on a n points (X_i, Y_i) , un modèle polynomial d'ordre $n - 1$ aura une adéquation parfaite ($SSE = 0$). Cependant, ce modèle ne sera pas utile si on veut prédire des valeurs futures. L'objectif de ce chapitre est présenter des méthodes pour sélectionner des modèles en se basant sur la précision de prédiction.

À cet effet, il est important de distinguer deux types d'erreur: taux d'erreur entraînement et taux d'erreur test.

Le taux d'erreur entraînement peut être évalué par SSE . Il mesure l'adéquation des modèles aux données. On l'utilise pour comparer l'adéquation de plusieurs modèles à un jeu de données.

Le taux d'erreur test évalue la précision d'une observation future. Dans la prochaine section, on présente quelques méthodes pour l'estimer.

2 Validation croisée

L'idée de la validation croisée est de partager le jeu de données en deux parties. La première partie est appelée jeu de données entraînement. Elle sert à estimer les paramètres du modèle. La deuxième partie est le jeu de données test. Elle sert à estimer le taux d'erreur test.

Revenons au jeu de données **Auto** de la library **ISLR** du logiciel R. Il est composé de $n = 392$ observations. On le partage en deux jeux de données. Le premier est composé des $n_1 = 275$ premières observations. Il sert à estimer les paramètres β . Le deuxième est composé des $n_2 = n - n_1 = 392 - 275 = 117$ dernières observations. Il sert à estimer le taux d'erreur test par

$$\frac{1}{n_2} \sum_{i=n_1+1}^{n_2} (Y_i - \hat{Y}_i)^2 = \frac{1}{117} \sum_{i=276}^{392} (Y_i - \hat{Y}_i)^2.$$

Si on considère le modèle 1, alors $\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1 X_i$ est la valeur prédite de Y_i avec $\hat{\beta}_0$ et $\hat{\beta}_1$ calculé à partir de $\{(Y_i, X_i), i = 1, \dots, 275\}$.

```
> n <- 392
> n1 <- 275
> entraînement <- 1:n1
> modele.1.entraînement <- lm(mpg ~ horsepower,
                             data=Auto, subset=entraînement)
> modele.2.entraînement <- lm(mpg ~ horsepower+log(horsepower),
                             data=Auto, subset=entraînement)
> modele.3.entraînement <- lm(mpg ~ horsepower+I(horsepower^2),
```

```

                                data=Auto,subset=entrainement)
> modele.4.entrainement <- lm(mpg ~ horsepower+I(horsepower^2)+I(horsepower^3),
                                data=Auto,subset=entrainement)
>
> test <- (n1+1):n
> Y.chapeau.1 <- predict(modele.1.entrainement,newdata=Auto[test,])
> Y.chapeau.2 <- predict(modele.2.entrainement,newdata=Auto[test,])
> Y.chapeau.3 <- predict(modele.3.entrainement,newdata=Auto[test,])
> Y.chapeau.4 <- predict(modele.4.entrainement,newdata=Auto[test,])
>
> Y <- Auto$mpg
>
> err.test.modele.1 <- mean((Y[test]-Y.chapeau.1)^2) ; err.test.modele.1
[1] 64.50982
> err.test.modele.2 <- mean((Y[test]-Y.chapeau.2)^2) ; err.test.modele.2
[1] 53.69829
> err.test.modele.3 <- mean((Y[test]-Y.chapeau.3)^2) ; err.test.modele.3
[1] 52.9343
> err.test.modele.4 <- mean((Y[test]-Y.chapeau.4)^2) ; err.test.modele.4
[1] 52.7879

```

C'est le modèle 4 qui semble avoir la meilleure prédiction. Cependant, la partition du jeux de données original en deux est arbitraire. La façon la plus facile de réduire ce aléa dû à la répartition du jeux de données en deux est de recommencer cette procédure un grand nombre de fois et de prendre des moyennes.

```

> if (!require("ISLR")) install.packages("ISLR")
> library("ISLR")#### Installer la librairie ISLR
>
> data(Auto)
>
> n <- 392
> n1 <- 275
>
> B <- 1000
> err.test.modele.1 <- NULL
> err.test.modele.2 <- NULL
> err.test.modele.3 <- NULL
> err.test.modele.4 <- NULL

```

```

>
> for(b in 1:B)
+ {
+   entrainement <- sample(n,n1)
+
+   modele.1.entrainement <- lm(mpg ~ horsepower,data=Auto,subset=entrainement)
+   modele.2.entrainement <- lm(mpg ~ horsepower+log(horsepower),
+                               data=Auto,subset=entrainement)
+   modele.3.entrainement <- lm(mpg ~ horsepower+I(horsepower^2),
+                               data=Auto,subset=entrainement)
+   modele.4.entrainement <- lm(mpg ~ horsepower+I(horsepower^2)+I(horsepower^3),
+                               data=Auto,subset=entrainement)
+
+   Y.chapeau.1 <- predict(modele.1.entrainement,newdata=Auto[-entrainement,])
+   Y.chapeau.2 <- predict(modele.2.entrainement,newdata=Auto[-entrainement,])
+   Y.chapeau.3 <- predict(modele.3.entrainement,newdata=Auto[-entrainement,])
+   Y.chapeau.4 <- predict(modele.4.entrainement,newdata=Auto[-entrainement,])
+
+   Y <- Auto$mpg
+
+   err.test.modele.1 <- c(err.test.modele.1,mean((Y[-entrainement]-Y.chapeau.1)^2))
+   err.test.modele.2 <- c(err.test.modele.2,mean((Y[-entrainement]-Y.chapeau.2)^2))
+   err.test.modele.3 <- c(err.test.modele.3,mean((Y[-entrainement]-Y.chapeau.3)^2))
+   err.test.modele.4 <- c(err.test.modele.4,mean((Y[-entrainement]-Y.chapeau.4)^2))
+ }
>
>
> mean(err.test.modele.1)
[1] 24.27115
> mean(err.test.modele.2)
[1] 19.81173
> mean(err.test.modele.3)
[1] 19.30965
> mean(err.test.modele.4)
[1] 19.42957

```

En fin de compte, le modèle 3 semble être le meilleur pour prédire des observations futures. Même si on répète l'opération plusieurs fois, on obtient le même résultat.

Un deuxième défaut des deux approches présentées est la précision des esti-

mateurs $\hat{\beta}$. En effet, nous sommes en train d'estimer β avec seulement 70% des données. Et donc la variance de l'estimateur $\hat{\beta}$ est relativement grande. Pour remédier à cela, pour $i = \dots, n$, on estime $\hat{\beta}$ avec toutes les observations sauf la $i^{\text{ème}}$. On calcule $\hat{y}_{i,(-i)}$. Le critère sera donc

$$VC_{(n)} = \frac{\sum_{i=1}^n (y_i - \hat{y}_{i,(-i)})^2}{n}.$$

On montre que

$$VC_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_i} \right)^2,$$

où h_i est le levier de l'observation i . Et par conséquent, on n'a pas besoin d'ajuster n régressions linéaires. Le code informatique est plus simple et surtout plus rapide.

```
> if (!require("ISLR")) install.packages("ISLR")
> library("ISLR")#### Installer la librairie ISLR
>
> data(Auto)
>
> CV.n <- function(modele)
+ {
+   residus <- modele$residuals
+   h <- hatvalues(modele)
+   resultat <- mean((residus/(1-h))^2)
+   return(resultat)
+ }
>
> modele.1 <- lm(mpg ~ horsepower, data=Auto)
> modele.2 <- lm(mpg ~ horsepower+log(horsepower),
+               data=Auto)
> modele.3 <- lm(mpg ~ horsepower+I(horsepower^2),
+               data=Auto)
> modele.4 <- lm(mpg ~ horsepower+I(horsepower^2)+I(horsepower^3),
+               data=Auto)
>
> CV.n(modele.1)
[1] 24.23151
> CV.n(modele.2)
```

```
[1] 19.70068
> CV.n(modele.3)
[1] 19.24821
> CV.n(modele.4)
[1] 19.33498
```

Le modèle 3 est encore une fois le meilleur pour prédire des observations futures.

Le calcul de $VC_{(n)}$ est direct dans le cas d'une régression linéaire. Cependant, pour d'autres modèles de régression dans la famille des régressions linéaires généralisés, ce calcul requière l'ajustement de n modèles de régression, ce qui peut être difficile d'un point de vue informatique. Pour cette raison, on peut considérer une autre version de validation croisée $VC_{(K)}$. Dans la pratique, on utilise $K = 5$ ou $K = 10$. On partage le jeu de données en K parties de tailles quasiment égales: $A_1, \dots, A_K$. Pour $k = 1, \dots, K$, on estime les paramètres β à partir de toutes les données sauf A_k et on utilise A_k pour estimer l'erreur avec

$$MSE_k = \frac{1}{n_k} \sum_{i \in A_k} \{y_i - \hat{y}_{i,(-k)}\}^2,$$

où n_k est la taille de A_k . On estime par la suite l'erreur test par :

$$VC_{(K)} = \frac{1}{K} \sum_{k=1}^K MSE_k.$$

```
> #####
> ## Calcul de CV.n et CV.k en utilisant la librairie boot ##
> #####
>
> if (!require("boot")) install.packages("boot")
> library("boot")#### Installer la librairie boot
>
> modele.glm.1 <- glm(mpg ~ horsepower,data=Auto)
> modele.glm.2 <- glm(mpg ~ horsepower+log(horsepower),
+                       data=Auto)
> modele.glm.3 <- glm(mpg ~ horsepower+I(horsepower^2),
+                       data=Auto)
> modele.glm.4 <- glm(mpg ~ horsepower+I(horsepower^2)+I(horsepower^3),
+                       data=Auto)
>
```

```

> ## CV.n
> cv.glm(Auto, modele.glm.1)$delta[1]
[1] 24.23151
> cv.glm(Auto, modele.glm.2)$delta[1]
[1] 19.70068
> cv.glm(Auto, modele.glm.3)$delta[1]
[1] 19.24821
> cv.glm(Auto, modele.glm.4)$delta[1]
[1] 19.33498
>
> ## CV.K (K=5)
> cv.glm(Auto, modele.glm.1, K=5)$delta[1]
[1] 24.26099
> cv.glm(Auto, modele.glm.2, K=5)$delta[1]
[1] 19.80128
> cv.glm(Auto, modele.glm.3, K=5)$delta[1]
[1] 19.24542
> cv.glm(Auto, modele.glm.4, K=5)$delta[1]
[1] 19.30565
>
> ## CV.K (K=10)
> cv.glm(Auto, modele.glm.1, K=10)$delta[1]
[1] 24.13985
> cv.glm(Auto, modele.glm.2, K=10)$delta[1]
[1] 19.92659
> cv.glm(Auto, modele.glm.3, K=10)$delta[1]
[1] 19.23274
> cv.glm(Auto, modele.glm.4, K=10)$delta[1]
[1] 19.46304

```

3 Régression régularisée

Considérons un modèle de régression

$$Y_i = \beta_0 + \beta_1 X_{i1} + \cdots + \beta_p X_{ip} + \epsilon_i; \quad i = 1, \dots, n; \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2).$$

On estime $\beta = (\beta_0, \beta_1, \dots, \beta_p)^\top$ en minimisant

$$l(\beta_0, \beta_1, \dots, \beta_p) = \sum_{i=1}^n \{Y_i - (\beta_0 + \beta_1 X_{i1} + \cdots + \beta_p X_{ip})\}^2.$$

En écriture matricielle, on a $l(\beta) = (Y - X\beta)^\top(Y - X\beta)$. L'estimateur obtenu vérifie

$$\hat{\beta} \sim \mathcal{N}\{\beta, \sigma^2(X^\top X)^{-1}\}.$$

On montre que $\hat{\beta}$ est sans biais et de variance minimale parmi les estimateurs sans biais. Cependant, rien ne garantit que $\hat{\beta}$ possède la plus petite erreur quadratique moyenne. Pour $j = 1, \dots, p$, l'erreur quadratique moyenne de β_j est

$$E[(\beta_j - \hat{\beta}_j)^2] = [E(\hat{\beta}_j) - \beta_j]^2 + \text{Var}(\hat{\beta}_j) = \text{biais}(\hat{\beta}_j)^2 + \text{Var}(\hat{\beta}_j).$$

La régression régularisée impose une pénalité à $l(\beta_0, \beta_1, \dots, \beta_p)$. On estime alors β en minimisant

$$l_\lambda(\beta_0, \beta_1, \dots, \beta_p) = l(\beta_0, \beta_1, \dots, \beta_p) + \lambda g(\beta_1, \dots, \beta_p),$$

où g est une fonction de pénalité connue et λ est un paramètre, appelé coefficient de pénalité, qui permet de contrôler l'impact de la pénalité imposée à $\hat{\beta}$. L'objectif de cette pénalité est de réduire la variance des estimateurs obtenus. Cependant, cette pénalité introduit un biais. Avec un choix judicieux de la fonction g et du paramètre λ , on diminue l'erreur quadratique moyenne des estimateurs et on obtient des modèles avec prédictions plus précises. Dans les sous-sections suivantes, on présente les régressions régularisées les plus populaires.

3.1 Régression Ridge

La régression ridge estime β en minimisant

$$l_\lambda^{(R)}(\beta_0, \beta_1, \dots, \beta_p) = \sum_{i=1}^n \{Y_i - (\beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip})\}^2 + \lambda \sum_{j=1}^p \beta_j^2.$$

En écriture matricielle, on a $l_\lambda^{(R)}(\beta) = (Y - X\beta)^\top(Y - X\beta) + \lambda \beta^\top \tilde{I}_p \beta$. Ici, $\tilde{I}_p$ est une matrice diagonale de taille $p+1$. Le premier élément de la diagonale de $\tilde{I}_p$ est égal à 0. Tous les autres éléments de la diagonale de $\tilde{I}_p$ sont égaux à 1. On obtient alors:

$$\hat{\beta}_\lambda^{(R)} = (X^\top X + \lambda \tilde{I}_p)^{-1} X^\top Y$$

et on en déduit que

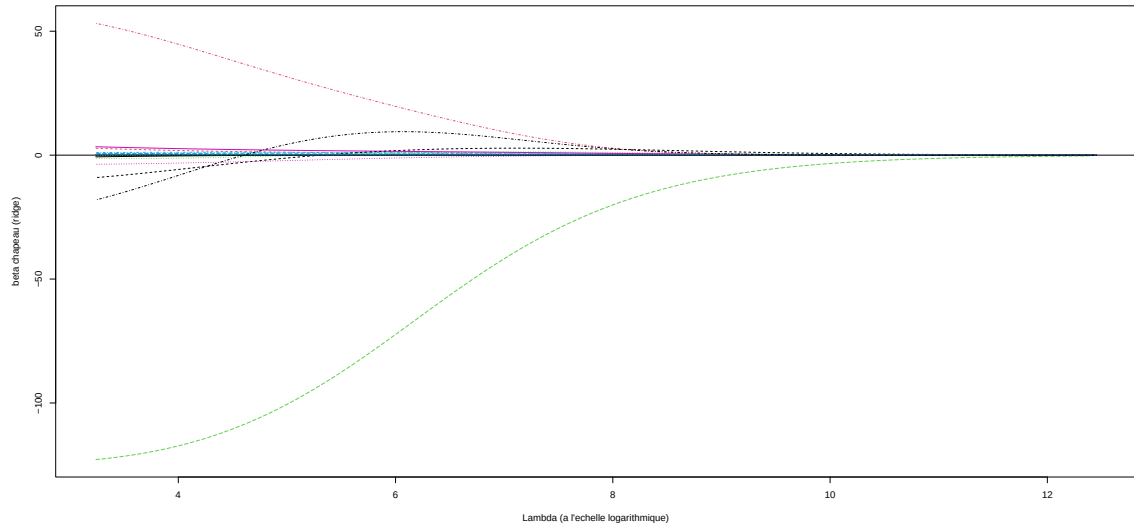
$$\begin{aligned} E[\hat{\beta}_\lambda^{(R)}] &= (X^\top X + \lambda \tilde{I}_p)^{-1} X^\top X \beta \neq \beta \\ \text{Var}[\hat{\beta}_\lambda^{(R)}] &= \sigma^2 (X^\top X + \lambda \tilde{I}_p)^{-1} X^\top X (X^\top X + \lambda \tilde{I}_p)^{-1} \end{aligned}$$

L'estimateur $\hat{\beta}_\lambda^{(R)}$ est donc biaisé, sauf si $\lambda = 0$.

Considérons maintenant le jeu de données `Hitters` de la library `ISLR` du logiciel R. La variable réponse est le salaire d'un joueur professionnel de base-ball au début de la saison 1987 en milliers de dollars. Les variables explicatives portent essentiellement sur les performances et les caractéristiques des joueurs. On considère un modèle linéaire avec toutes les variables explicatives, sans interactions, et on estime les paramètres $\beta_j, j = 1, \dots, 19$ en utilisant une régression ridge avec différentes valeurs de λ .

```
> if (!require("ISLR")) install.packages("ISLR")
> library("ISLR")#### Installer la librairie ISLR
>
> data(Auto)
>
> if (!require("glmnet")) install.packages("glmnet")
> library("glmnet") #### Installer la librairie glmnet
>
> #####
> ## Preparation des donnees ##
> #####
>
> data(Hitters)
> Hitters <- Hitters[!is.na(Hitters$Salary),]
> x <- model.matrix(Salary ~ ., Hitters)[, -1]
> y <- Hitters$Salary
>
> #####
> ## Regression Ridge avec differentes valeurs de lambda ##
> #####
>
> reg.ridge <- glmnet(x, y, alpha = 0)
> beta.chapeau.ridge <- t(coef(reg.ridge)[-1,])
> lambda.ridge <- reg.ridge$lambda
> log.lambda.ridge <- log(lambda.ridge)
> matplot(log.lambda.ridge,beta.chapeau.ridge,type="l",
+          xlab="Lambda (a l'echelle logarithmique)",ylab="beta chapeau (ridge)")
> abline(h=0)
```

Et on obtient



Lorsque $\lambda = 0$, on a $\hat{\beta}_{\lambda}^{(R)} = \hat{\beta}$ et lorsque $\lambda \rightarrow \infty$, les estimateurs de $\beta_j, j = 1, \dots, p$ tendent tous vers 0 et l'estimateur de $\hat{\beta}_{0,\lambda}^{(R)}$ tend vers la moyenne $\bar{Y}$. Pour illustrer l'effet de la régression ridge, on fixe $\lambda = e^6$ et on compare $\sum_{j=1}^p \hat{\beta}_j^2$ à $\sum_{j=1}^p [\hat{\beta}_{j,\lambda}^{(R)}]^2$.

```
> lambda.ridge <- exp(6)
> reg.ridge.2 <- glmnet(x, y, alpha = 0, lambda=lambda.ridge)
> reg.ridge.3 <- glmnet(x, y, alpha = 0, lambda=0)
>
> sum((coef(reg.ridge.2)[-1])**2)
[1] 5720.707
> sum((coef(reg.ridge.3)[-1])**2)
[1] 18468.89
```

On trouve $\sum_{j=1}^p \hat{\beta}_j^2 = 18468.89$ et $\sum_{j=1}^p [\hat{\beta}_{j,\lambda}^{(R)}]^2 = 5720.707$. On parle d'un phénomène de rétrécissement.

Généralement, la régression ridge diminue considérablement le taux d'erreur test. Par conséquent, elle croît la précision des prédictions. Estimons le taux d'erreur test avec $\lambda = e^6$ et comparons ce taux d'erreur avec celui de la régression linéaire standard, qui correspond à $\lambda = 0$.

```

> x.entrainement <- x[1:200,]
> y.entrainement <- y[1:200]
> reg.ridge.2.entrainement <- glmnet(x.entrainement,y.entrainement,
+                                   alpha=0,lambda=lambda.ridge)
>
> x.test <- cbind(rep(1),x[201:263,])
> y.test <- y[201:263]
> y2.predites <- x.test%%as.vector(coef(reg.ridge.2.entrainement))
> err.test.2 <- mean((y.test-y2.predites)**2); err.test.2
[1] 91274.42
>
> reg.ridge.3.entrainement <- glmnet(x.entrainement,y.entrainement,
+                                   alpha=0,lambda=0)
> y3.predites <- x.test%%as.vector(coef(reg.ridge.3.entrainement))
> err.test.3 <- mean((y.test-y3.predites)**2); err.test.3
[1] 124716.2

```

La diminution du taux d'erreur test est considérable (près de 27%).

3.2 Régression Lasso

La régression Lasso estime β en minimisant

$$l_{\lambda}^{(L)}(\beta_0, \beta_1, \dots, \beta_p) = \sum_{i=1}^n \{Y_i - (\beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip})\}^2 + \lambda \sum_{j=1}^p |\beta_j|.$$

Il n'existe pas d'expression explicite pour l'estimateur obtenu $\hat{\beta}_{\lambda}^{(L)}$. Cependant, il existe de nos jours plusieurs algorithmes d'optimisation pour minimiser $l_{\lambda}^{(L)}(\beta_0, \beta_1, \dots, \beta_p)$.

Une caractéristique intéressante de la régression Lasso est que pour les grandes valeurs de λ , certains termes $\hat{\beta}_j$ deviennent identiquement égaux à 0. Cet aspect permet une méthode très efficace de sélection de modèle.

Encore une fois, lorsque $\lambda = 0$, on a $\hat{\beta}_{\lambda}^{(L)} = \hat{\beta}$. Lorsque λ croît, le biais de $\hat{\beta}_{\lambda}^{(L)}$ diminue alors que sa variance croît.

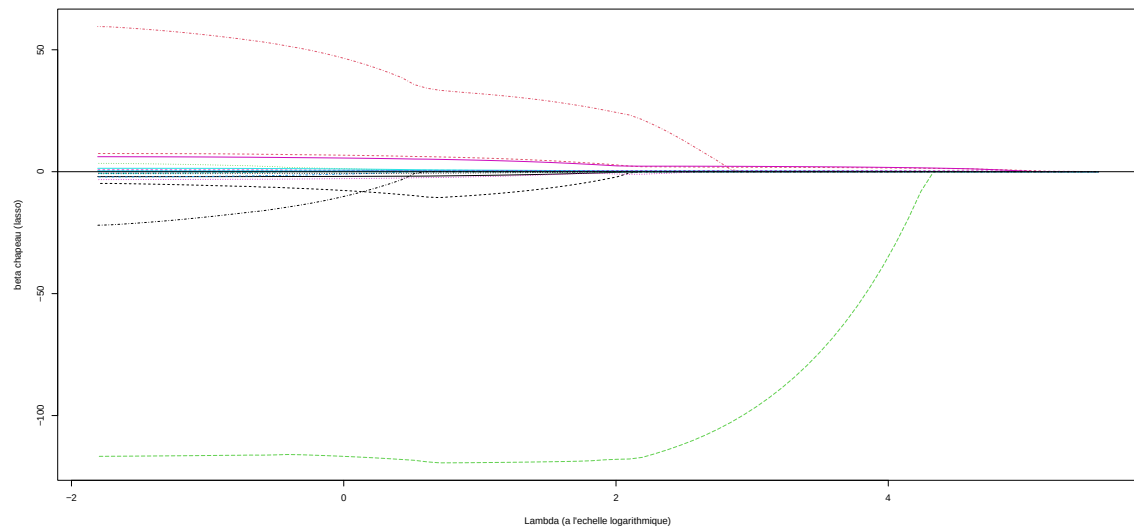
```

> reg.lasso <- glmnet(x, y, alpha = 1)
> beta.chapeau.lasso <- t(coef(reg.lasso)[-1,])
> lambda.lasso <- reg.lasso$lambda
> log.lambda.lasso <- log(lambda.lasso)

```

```
> matplot(log.lambda.lasso,beta.chapeau.lasso,type="l",
+         xlab="Lambda (a l'echelle logarithmique)",ylab="beta chapeau (lasso)")
> abline(h=0)
```

Et on obtient



On se fixe $\lambda = e^2$ et on calcule $\hat{\beta}_\lambda^{(L)}$.

```
> ## On se fixe un lambda : log(lambda)=2
>
> lambda.lasso <- exp(2)
>
> reg.lasso.2 <- glmnet(x, y, alpha = 1, lambda=lambda.lasso)
> coef(reg.lasso.2)
20 x 1 sparse Matrix of class "dgCMatrix"
              s0
(Intercept) 16.56691118
AtBat      -0.28054898
Hits       2.83421809
HmRun      .
Runs       .
RBI        .
```

Walks	2.43602385
Years	-2.17005788
CAtBat	.
CHits	.
CHmRun	0.07794600
CRuns	0.25027060
CRBI	0.41108058
CWalks	-0.01322173
LeagueN	24.41162017
DivisionW	-117.77228136
PutOuts	0.24305524
Assists	.
Errors	-0.89001203
NewLeagueN	.

Et on trouve que plusieurs coefficients $\hat{\beta}_{j,\lambda}^{(L)}$ sont identiquement égaux à zéro. Ceci nous procure une méthode intuitive et naturelle de sélection de modèles.

3.3 Sélection du paramètre λ

Il existe quelques méthodes pour sélectionner le paramètre λ . La méthode la plus intuitive est celle basée sur le taux d'erreur test. Ce taux d'erreur est estimé par validation croisée pour plusieurs valeurs de λ . On choisit par la suite la valeur de λ qui minimise le taux d'erreur test.

Pour la régression ridge, on a :

```
> grille.lambda.ridge <- exp(seq(.5,14,0.1))
> reg.ridge.select <- cv.glmnet(x, y, alpha = 0, lambda=grille.lambda.ridge)
> log(reg.ridge.select$lambda.min)
[1] 2.2
```

et donc, on choisit $\lambda = e^{2.2}$.

Et pour la régression lasso, on a :

```
> plot(reg.ridge.select)
> ## On selectionne un lambda en se basant sur la validation croisee
>
> grille.lambda.lasso <- exp(seq(-3,6,0.1))
>
> reg.lasso.select <- cv.glmnet(x,y,alpha=1,lambda=grille.lambda.lasso)
> log(reg.lasso.select$lambda.min)
[1] 3
```

et donc, on choisi $\lambda = e^3$.